Inheritance, Overriding and Polymorphism

10 min read Updated 28 Jul 2026 Topic 7 of 65

Bookmark Mark done

Why it matters

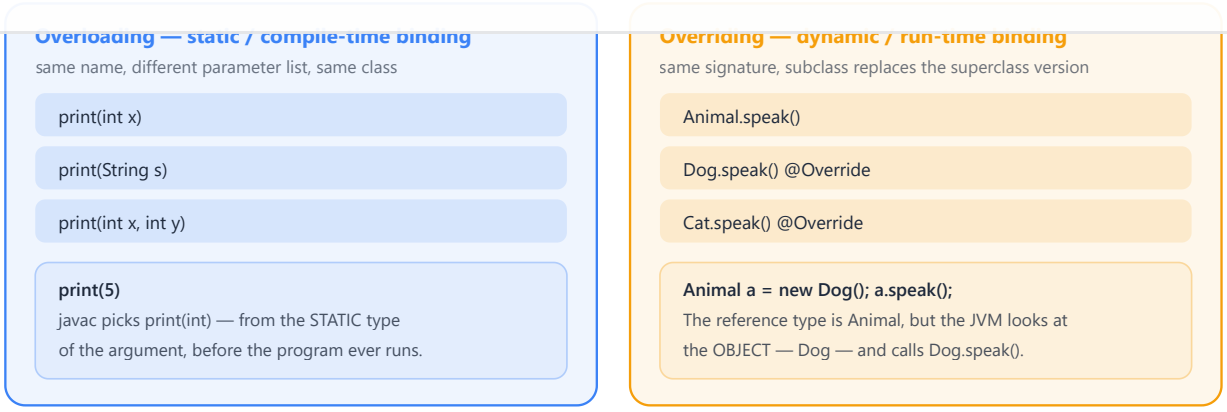
Polymorphism is the mechanism that makes every framework you use possible. Spring can call your controller, JUnit can call your test, a `List` can be an `ArrayList` or a `LinkedList` without the calling code caring — all of it is one JVM feature: the method that runs is chosen by the object, not by the variable that names it.

It is also the richest source of trick questions in a Java interview, because overloading and overriding look similar and behave completely differently. Get the distinction crisp and a whole family of "what does this print?" questions becomes trivial.

The concept, simply

Inheritance lets a class extend another, acquiring its non-private members and adding or replacing behaviour. Java allows single inheritance of classes — one `extends` — and multiple inheritance of interfaces.

Overriding is a subclass supplying its own version of an inherited method with the same signature. **Overloading** is several methods in the same class sharing a name but differing in parameters. They are unrelated ideas with confusingly similar names.



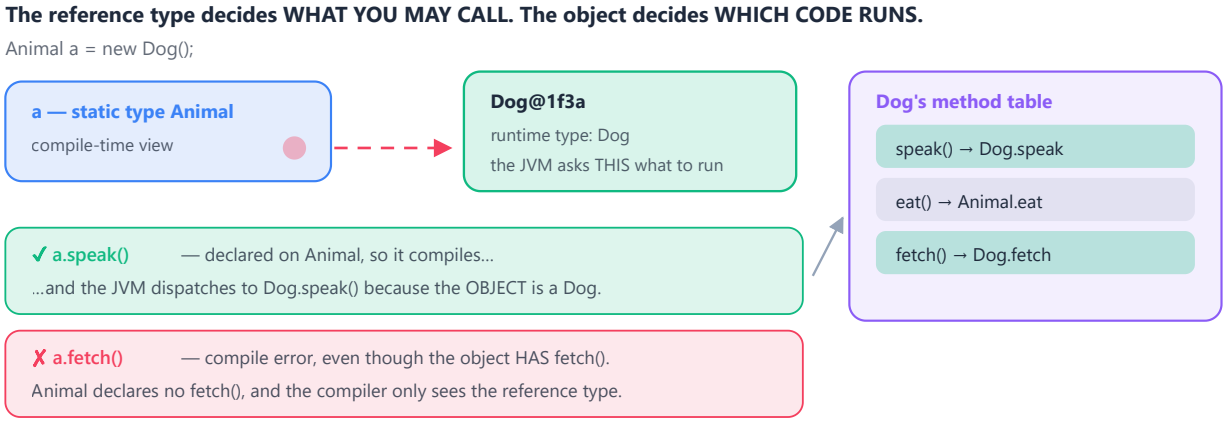
The one-line summary: overloading is a compile-time decision based on the declared types; overriding is a run-time decision based on the actual object.

The distinction has a precise name. Overloading is **static binding** — `javac` picks the target method and burns it into the bytecode. Overriding is **dynamic binding** — the bytecode names a method, and the JVM decides at the call which implementation to run by looking at the object.

How it works

Two types are in play at every call, and keeping them apart solves most puzzles:

- The **static (declared) type** — what the variable is declared as. It decides *what you are allowed to call*.
- The **runtime type** — what the object actually is. It decides *which implementation runs*.



The rules for a valid override are worth knowing exactly, because interviewers test the edges:

Aspect	Rule
Signature	Must match exactly — name and parameter types. Different parameters = overloa
Return type	Same, or a **covariant** subtype (an override may return `Dog` where the parent retu
Access	May widen, never narrow. `protected` → `public` is fine; `public` → `protected`
Checked exceptions	May throw fewer or narrower ones, never new or broader ones. Unchecked excepti
Cannot override	`final`, `static` and `private` methods, and constructors.

Every one of those restrictions serves the same principle — the **Liskov substitution principle**. Code written against the parent must keep working when handed a child. Narrowing access or throwing a new checked exception would break a caller that compiled fine against the parent, so the language forbids it.

Code examples

Polymorphism doing its job — one loop, three behaviours, no `if`:

JAVA

Copy

```
abstract class Animal {
    abstract String speak();
    void describe() { System.out.println(getClass().getSimpleName() + " sa
}
class Dog extends Animal { String speak() { return "Woof"; } }
class Cat extends Animal { String speak() { return "Meow"; } }
```

```
for (Animal a : animals) a.describe();
// Dog says Woof / Cat says Meow / Cow says Moo
//
// describe() was compiled once against Animal and has never heard of Cow.
// Adding a fourth animal requires no change here at all — that is the pay
```

The trick question this topic exists for. Fields and static methods are *not* polymorphic:

JAVA Copy

```
class Parent {
    String label = "parent-field";
    String instanceMethod() { return "parent-instance"; }
    static String staticMethod() { return "parent-static"; }
}
class Child extends Parent {
    String label = "child-field";                    // HIDES, does not
    @Override String instanceMethod() { return "child-instance"; }
    static String staticMethod() { return "child-static"; }   // HIDES
}

Parent p = new Child();
System.out.println(p.label);           // parent-field    ← static type w
System.out.println(p.instanceMethod()); // child-instance  ← runtime type
System.out.println(p.staticMethod());   // parent-static   ← static type w

// Only INSTANCE METHODS are dynamically dispatched. Fields and static
// methods are resolved from the declared type at compile time. This is
// called hiding, and it is a reason to avoid re-declaring either.
```

Overload resolution surprises, all decided before the program runs:

JAVA Copy

```
static void f(Object o) { System.out.println("Object"); }
static void f(String s) { System.out.println("String"); }
```

```
// Add a third overload and the last line stops compiling:
// static void f(Integer i) { }
// f(null);  → "reference to f is ambiguous" — String and Integer are
//             equally specific and neither is a subtype of the other.
```

And `@Override` earning its keep:

JAVA Copy

```
class Base { boolean equals(Base other) { return true; } }   // ✗ silent b
// this OVERLOADS Object.equals(Object) instead of overriding it, so HashM
// and every library that calls equals(Object) ignores it entirely.

class Fixed {
    @Override                          // ✓ compiler now checks the claim
    public boolean equals(Object o) { … }
}
// @Override is optional to the JVM and close to mandatory in practice:
// it turns a silent behavioural bug into a compile error.
```

Under the hood

Dynamic dispatch is implemented with a **virtual method table**. Each class has a table mapping method slots to implementations; a subclass inherits its parent's table and overwrites the slots it overrides. A virtual call compiles to `invokevirtual`, which at run time reads the object's class pointer, finds the table, and jumps to whatever is in the slot. That is one indirection — cheap, but not free.

Except the JIT usually removes even that. If profiling shows a call site has only ever seen one receiver type — the **monomorphic** case, which is the overwhelming majority in real code — the JIT compiles a direct call guarded by a class check, and then inlines it. This is why "virtual calls are slow, mark everything `final` for performance" is obsolete advice:

The four `invoke` instructions are worth being able to name: `invokevirtual` for ordinary instance methods, `invokeinterface` for interface calls, `invokestatic` for statics, and `invokespecial` for constructors, `private` methods and explicit `super` calls — the three cases where dispatch must *not* be virtual.

Common mistakes

- **Confusing overloading with overriding.** Different parameters means a new method, and the compiler will happily let you think you overrode something.
- **Omitting `@Override`.** It is the cheapest bug prevention in Java. Always write it.
- **Expecting fields to be polymorphic.** They are resolved from the declared type. Re-declaring a field in a subclass hides it and is almost always a mistake.
- **Thinking static methods can be overridden.** They are hidden, and dispatched from the declared type. Calling a static through an instance reference is legal and misleading — do not.
- **Extending for code reuse.** Inheritance is an "is-a" claim that also inherits every future change to the parent. Favour composition unless substitutability is genuinely true.
- **Overriding `equals` with the wrong parameter type.** `equals(MyType)` overloads rather than overrides — the exact bug `@Override` catches.

Interview lens

> **Q: "Difference between overloading and overriding?"**
> A: Overloading is several methods in one class with the same name and different parameter lists; the compiler picks one from the arguments' declared types, so it is static binding. Overriding is a subclass replacing an inherited method with the identical signature; the JVM picks the implementation from the actual object at run time, so it is dynamic binding. Only overriding is polymorphism.

A: No. Redeclaring a static method in a subclass *hides* it. Static methods belong to the class and are resolved from the declared type at compile time, so there is no dispatch on the object. `Parent p = new Child(); p.staticMethod();` calls the parent's version.

Q: "Are fields polymorphic?"

A: No. Field access is resolved from the static type. If a subclass declares a field with the same name it hides the parent's, and which one you see depends on the declared type of the reference, not the object. It is a strong argument for keeping fields private.

Q: "What are the rules for a valid override?"

A: Same name and parameter types; return type identical or covariant; access may widen but never narrow; checked exceptions may be narrowed or dropped but never broadened; and `final`, `static` and `private` methods cannot be overridden. All of it follows from Liskov substitution — code compiled against the parent must still work with a child.

Q: "Why does Java not support multiple inheritance of classes?"

A: To avoid the diamond problem for *state*. If a class inherited from two parents that both defined a field, the object would have two copies and no rule about which to use. Java allows multiple interface inheritance because interfaces carry no instance state — and where default methods create an ambiguity, the compiler forces you to resolve it explicitly.

Q: "How is dynamic dispatch implemented, and is it slow?"

A: Each class has a virtual method table; a subclass inherits its parent's and overwrites overridden slots. `invokevirtual` reads the object's class pointer and jumps through the table — one indirection. In practice it is usually free, because

most call sites are monomorphic and the JIT devirtualises and inlines them behind a guard, deoptimising if a second type appears.

Q: "When would you choose composition over inheritance?"

A: Almost by default. Inheritance couples you to the parent's implementation forever and is only honest when the subclass genuinely is substitutable. If I only want to reuse code, composition — hold an instance and delegate — gives the same reuse without the coupling, works with `final` classes, and can be changed at run time. The classic counter-example is `java.util.Stack` extending `Vector`, which leaks methods that let you break the stack invariant.

Cheat-card

- **Overload** = same name, different parameters, **compile-time**, static type.
- **Override** = same signature in a subclass, **run-time**, actual object.
- Static type = what you may call. Runtime type = which code runs.
- **Not polymorphic**: fields, `static` methods, `private` methods → hidden, not overridden.
- Override rules: covariant return, **widen access only**, no broader checked exceptions.
- Always write `@Override` — turns a silent overload into a compile error.
- Single class inheritance, multiple interface inheritance — the split exists because interfaces have no state.
- vtable + `invokevirtual`; JIT devirtualises monomorphic sites, so `final`-for-speed is obsolete.
- Prefer **composition** unless "is-a" is genuinely true.

QUICK CHECK — no marks, just for you.

1. Parent p = new Child(); Which of p.label (a field), p.instanceMethod() and p.staticMethod() resolve to the Child version?

- ◯ Only the field, because fields are looked up on the instance
- ◯ None — the reference type is Parent
- ◯ Only p.instanceMethod() — fields and static methods are resolved from the declared type

Check answer

2. Which change to an overriding method will NOT compile?

- ◯ Declaring a broader checked exception than the parent declared
- ◯ Declaring no checked exceptions at all
- ◯ Returning Dog where the parent returned Animal
- ◯ Widening access from protected to public

Check answer

3. Why is 'mark methods final for performance' obsolete advice on a modern JVM?

- ◯ Because final methods are actually slower than virtual ones
- ◯ Because the JIT devirtualises monomorphic call sites and inlines them behind a guard, deoptimising if a second type appears
- ◯ Because all methods are final by default since Java 8
- ◯ Because the JVM no longer uses virtual method tables

Check answer

Previous
Classes, Objects and Constructors

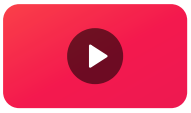
Next
Abstract Classes vs Interfaces

LEARN MORE

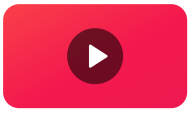
ARTICLES & DOCS

- Inheritance — The Java Tutorials
  docs.oracle.com
- Method overriding and hiding (JLS §8.4.8)
  docs.oracle.com

VIDEOS



"inheritance and overriding" — video explainers
YouTube



inheritance and overriding interview questions
YouTube

Hand-picked references for this lesson.